

Operators

In Python, operators are the **basic building blocks** of the programming language and are crucial for precisely defining how data is processed and linked within a program. They enable values and variables to communicate in a clearly defined manner and give the program the ability to perform mathematical operations, compare values, check conditions, and make decisions based on logical or functional requirements. By using operators, Python converts abstract data into concrete results, gives the programmer control over the execution flow, and enables efficient information processing and precise data management. Operators define the structure of expressions and the behavior of programs, making the code clear, predictable, and professionally organized, which is crucial for developing complex and maintainable software solutions.

Their application enables consistent, understandable, and easy-to-maintain program logic, with each operator, although simple in form, playing an important role in precisely determining the functionality and dynamics of the program. In this way, operators not only enable the manipulation of data, but also lay the foundation for professional, robust, and structured programming, where every line of code has a clearly defined purpose and contributes to the overall efficiency and reliability of the **software solution**.

Arithmetic operators:

Operator	Meaning	Example
+	Addition	$3 + 2 = 5$
-	Subtraction	$5 - 2 = 3$
*	Multiplication	$4 * 3 = 12$
/	Division	$6 / 2 = 3$
//	Integer division	$7 // 2 = 3$
%	Division remainder	$7 \% 2 = 1$
**	Exponent	$2 ** 3 = 8$

They are used to perform **basic mathematical operations** with numbers. Python supports addition, subtraction, multiplication, division, division remainders, and exponentiation. We use them when we want to calculate values of variables or expressions within a program. For example, $x = 5 + 3$ adds the numbers and stores the result in x , while $y = 10 \% 3$ gives the remainder when ten is divided by three. This allows us to perform all basic arithmetic calculations directly in the code.

Assignment operators:

Operator	Meaning	Example
=	Simple assignment	x = 5
+=	Addition and assignment	x += 2
-=	Subtraction and assignment	x -= 2
*=	Multiplication and assignment	x *= 2
/=	Division and assignment	x /= 2
//=	Integer division and assignment	x //= 2
%=	Modulo and assignment	x %= 2
**=	Exponent and assignment	x **= 2

Used to assign a value to a variable. For example, `x = 10` means that the variable `x` now holds the number 10. There are also shorter forms such as `x += 5`, which increase the variable's value by a certain amount. In this way, we can easily change the values of variables while the program is running

Comparison operators:

Operator	Meaning	Example
<code>==</code>	Equal	<code>5 == 5 → True</code>
<code>!=</code>	Not equal	<code>5 != 3 → True</code>
<code>></code>	Greater than	<code>5 > 3 → True</code>
<code><</code>	Less than	<code>3 < 5 → True</code>
<code>>=</code>	Greater than or equal to	<code>5 >= 5 → True</code>
<code><=</code>	Less than or equal to	<code>3 <= 5 → True</code>

They are used to compare the values of two variables or expressions. They return **True** or **False** depending on whether the condition is met. For example, `5 > 3` checks if five is greater than three, and the result is **True**...

Logic operators:

Operator	Meaning	Example
and	Logical AND	True and False → False
or	Logical OR	True or False → True
not	Logical NOT	not True → False

Logical operators are used to combine or negate conditions and always return a value of **True** or **False**.

Bitwise operators:

Operator	Meaning	Example
&	AND	5 & 3 → 1
	OR	5 3 → 7
^	XOR	5 ^ 3 → 6
~	NOT (inverts the bits)	~5 → -6
<<	Shift left	2 << 1 → 4
>>	Shift right	4 >> 1 → 2

Bitwise operators work directly on the **binary representations of numbers**, that is, on their individual bits. For example, 5 & 3 performs an AND operation between the binary values of 5 (101) and 3 (011), resulting in 1.

Membership operators:

Operator	Meaning	Example
in	Is an element in the sequence	'a' in 'apple' → True
not in	Is an element not in the sequence	'b' not in 'apple' → True

The membership operator checks whether a certain value is present within a data collection, such as a list, array, or set, and returns **True** or **False**.

Identity operators:

Operator	Meaning	Example
is	Is it the same object	x is y
is not	Is it not the same object	x is not y

Identity operators check whether two variables actually point to the same location in memory, not just whether they have the same value.

QUESTION: What will the print functions display at the end of the following program?

```
x = -3
x //= 2
print(x)
y = -1
y **= 2
print(bool(y and []))
z = 7
print(z >> 1)
```

CONTROL FLOW

Control flow represents a fundamental mechanism in every programming language, including Python. While variables and operators allow data to be stored and manipulated, control flow determines the order in which different parts of a program are executed. Without control flow, a program would simply be a sequence of instructions executed strictly from start to finish, with no ability to make decisions or repeat certain sections of code. Through branching statements and loops, a program gains flexibility, becomes interactive, and is capable of solving more complex problems. Branching, or conditional statements, allows different sequences of code to execute depending on whether a condition is met. In this way, a program can “decide” which path to follow next, making it smarter and more adaptable. Loops, on the other hand,

allow a section of code to repeat as long as a certain condition holds or until a predefined number of iterations is completed. This eliminates the need to repeat identical instructions manually and improves efficiency, which is essential in programming. By combining **branching and loops**, a programmer gains a powerful tool to shape the execution flow and implement complex logic. Control flow, therefore, is what separates simple sequences of instructions from true programs that make decisions, respond to different situations, and perform tasks in an organized and efficient manner.

In this chapter, we will explore the basic mechanisms of control flow in Python, starting with if conditional statements, moving through different types of loops, and concluding with special statements that further influence a program's behavior during execution.

Conditional Statements

In programming, we often encounter situations where a program needs to respond differently under different circumstances. Python provides this control over the flow of execution through conditional statements: if, elif, and else. These statements allow certain parts of the code to run only when specific conditions are met, making the program dynamic and adaptable. The if statement represents the basic form of conditional execution.

The syntax is straightforward. After the keyword `if`, the condition is specified, and the block of code that executes if the condition is true is written indented below. For example, if we want to check whether a number is greater than zero:

Example 2.1:

```
num = 10
if num > 0:
    print("Number is positive")
```

In this example, the message will be displayed only if the number is greater than zero. If the condition is not met, Python simply skips that block and continues executing the program.

INDENTATION:

In Python, indentation is not just a matter of style—it is mandatory. Python uses spaces or tabs to define the block of code that belongs to a specific condition. All lines indented under a control statement are considered part of that block.

The else statement allows you to define an alternative flow of execution when the preceding if condition is not met. The keyword else is placed at the same indentation level as the if, and the block of code beneath it executes when the if condition evaluates to False.

Example 2.2:

```
num = -3
if num > 0:
    print("Number is positive")
else:
    print("Number is negative")
```

In this case, since the number is less than zero, the else block will execute, and the program will display a message about the negative number. Sometimes, we want to check multiple conditions one after another. In such cases, we use the elif statement (short for "else if"). It allows adding multiple branches in a program, while else is used to cover all remaining cases.

Example 2.3:

```
num = 0
if num > 0:
    print("Number is positive")
elif num < 0:
    print("Number is negative")
else:
    print("Number is zero")
```

In this example, Python first checks if `num > 0`. If the condition is false, it moves on to `elif num < 0`. If that condition is also not met, the `else` block is executed. It is important to note that only the first true condition is executed, and the rest are skipped.

RULES AND RECOMMENDATIONS:

Each if statement can have zero or more elif branches, but at most one else. Conditions can be simple (e.g., `number > 0`) or complex. It is good practice to organize conditions from the most specific to the most general, keeping the program clear and efficient. The else block is optional but often useful for handling unexpected cases.

PROBLEM: Write a Python program that asks the user to enter an integer and then displays an appropriate message indicating whether the number is positive, negative, or zero using if, elif, and else statements. Afterwards, the program should also check and display whether the number is even or odd, making proper use of indentation so that Python correctly recognizes which lines belong to which code blocks.

Loops

In Python, loops allow a block of code to be executed multiple times, making it easier to repeat tasks and automate processes without the programmer having to manually duplicate code. There are two main types of loops: while and for. A while loop executes a block of code as long as a given condition is true, checking the condition before each iteration, while a for loop is used to iterate over a sequence of elements or a generated range of numbers, executing the block of code for each element in the sequence. Proper indentation is crucial because Python uses spaces to determine which lines belong to the loop. Every line within the loop must be indented to be part of the corresponding block, whereas lines that are not indented do not belong to the loop and will execute only once. Additionally, loops can be combined with the break and continue statements, which allow for stopping the loop or skipping an iteration, providing more flexible control of the flow.

While Loop

It is used when we want to **execute a block of code as long as a certain condition is true**. This means the loop checks the condition before each iteration and continues to execute the block while the condition remains true. If the condition is never true, the block of code will not execute even once. For this reason, it is important to ensure that the condition eventually changes; otherwise, an infinite loop occurs, which never ends.

Example 2.4:

```
number = 1
while number <= 5:
    print("Number right now:", number)
    number+=1
```

In the shown example, the program starts with the variable number set to 1, and the while loop checks whether this number is less than or equal to 5. As long as the condition is true, the block of code within the loop executes, printing the current value of the number and

increasing it by 1, ensuring that the loop eventually ends. When the number becomes greater than 5, the condition is no longer satisfied, the loop stops executing, and the program continues. The result is the output of numbers from 1 to 5. Proper indentation of the code within the loop indicates which lines belong to the block, and updating the variable used in the condition is crucial to prevent creating an infinite loop.

For Loop

It is used when we want to execute a block of code for each element in a set of values or within a predefined range of numbers. The loop goes through all elements of a sequence, list, or the output of the `range()` function, executing the block of code for each element. Proper indentation is crucial so that Python can recognize which lines belong to the loop.

Example 2.5:

```
for i in range(1, 6):  
    print("Number right now:", i)
```

In this example, the loop uses the `range(1, 6)` function, which generates a sequence of numbers from 1 to 5, with each number temporarily assigned to the variable `i`. For each number in this range, the block of code inside the

loop executes, printing the current value of `i`. The loop automatically moves to the next element in the sequence until it reaches the end, after which it **stops executing**. The result is the output of numbers from 1 to 5.

Break and Continue

When working with loops, there is often a need to control their flow in specific ways, and this is where the **break** and **continue** statements come in. The **break** statement stops the execution of the loop as soon as a certain condition is met, and the program continues with the statement following the loop. In contrast, **continue** skips the current iteration and immediately moves to the next condition check, without terminating the entire loop.

Example 2.6:

```
for i in range(1, 8):
    if i == 3:
        continue
    if i == 6:
        break
    print("Number right now:", i)
print("Loop exited.")
```

In this example, the loop iterates through the numbers from 1 to 7, but when the variable *i* reaches 3, the `continue` statement is triggered, skipping the printing of that number and immediately moving to the next iteration. When *i* becomes 6, the `break` statement executes, immediately terminating the entire loop, and the program continues with the statement following it. As a result, the output is the numbers 1, 2, 4, and 5, with 3 being skipped and 6 marking the end of the loop's execution.

PROBLEM: Write a program that asks the user to enter multiple integers, one at a time. The program should continue accepting input until the user enters the number 0, at which point the loop is terminated using the break statement. If the user enters a negative number, it should be skipped using the continue statement, without processing or printing it. Positive numbers should be displayed on the screen immediately after being entered. When the loop ends, a message should appear on the screen indicating that the input has been completed.